Analiza Algoritmilor
Tema 2

18 decembrie 2012

Problema 1 – 0.7p

O expresie aritmetica complet parantezata este definita prin urmatorii constructori de baza:

0, 1, x, [e1+e2], [e1*e2], [-e2]

(2 constructori nulari, 1 constructor extern, 3 constructori interni. Sa notam cu ***E*** acest tip de date.)

De asemenea, se definesc operatorii:

eval(e, n): E x N -> N

(E1) eval(0, n) = 0
(E2) eval(1, n) = 1
(E3) eval(x, n) = n
(E4) eval([e1+e2], n) = eval(e1, n) + eval(e2, n)
(E5) eval([e1*e2], n) = eval(e1, n) * eval(e2, n)
(E6) eval([-e1], n) = -eval(e1, n)x

subst(e, f): E x E -> E

(S1) subst(0, f) = 0
(S2) subst(1, f) = 1
(S3) subst(x, f) = f
(S4) subst([e1+e2], f) = [subst(e1,f) + subst(e2, f)]
(S5) subst([e1*e2], f) = [subst(e1,f) * subst(e2, f)]
(S6) subst([-e], f) = [-subst(e, f)]

Sa se demonstreze prin inductie structurala ca pentru orice 2 expresii aritmetice ***e, f*** din ***E*** si ***n*** din ***N***, proprietatea urmatoare este adevarata:

P(e) = eval(subst(e, f), n)= eval(e, eval(f, n))

Problema 2 – 1.7p

Pentru tipul ***LIST*** definit prin constructorii de baza:

[] : -> LIST
[a] : T -> LIST
a::x (sau cons(a, x)) : T X LIST -> LIST (adica ***a*** adaugat la inceputul listei ***l***)

a) Definiti functiile:

member(a, l) – verifica apartenenta elementului ***a*** in lista ***l***

maxelem(l), minelem(l) – intorc elementul maxim, respectiv minim din lista ***l***
remove(l, a) : LIST x T -> LIST - intoarce o lista rezultata din eliminarea elementelor ***a*** din lista ***l***
double(l, a) : LIST x T -> LIST - intoarce o lista rezultat din dublarea elementelor ***a*** din ***l*** (fiecare ***a*** devine ***aa***)
count(l, a) – numara aparitiile elementului ***a*** in lista ***l***
size(l) – intoarce numarul de elemente din lista ***l***

Folosind recursivitatea si inductia structurala, demonstrati urmatoarele proprietati valabile pentru orice ***l*** de tip ***LIST***:

b) 2 * count(l, a) + size(remove(l, a)) == size(double(l, a))

c) member(a, l) -> (minelem(l) <= a <= maxelem(l)) ^ (member(a,reverse(l)))

Problema 3 – 0.6p

Utilizand invarianti la ciclare, demonstrati corectitudinea algoritmului lui Euclid, de mai jos, folosit pentru calculul cmmdc.

```
cmmdc(a, b)
    while (b != 0)
       t = b
       b = a mod t
       a = t
    return a
```

Problema 4 – 1p

Considerand un max-heap binar, cu operatiile INSERT si DELETE_MAX, pentru care se cere:

a) Care este complexitatea celor doua operatii in cazul cel mai defavorabil? Justificati.

b) Definiti o functie potential pentru max-heap astfel incat costurile amortizate ale celor doua operatii sa fie cat mai mici (DELETE_MAX sa fie O(1)).

c) Demonstrati prin metoda potentialului care sunt costurile amortizate ale celor doua operatii.

Problema 5 – 1p

Se considera un contor binar pe ***k*** biti care implicit are doar operatia INCREMENT (incrementeaza valoarea sa plecand de la 0).

a) Presupunand ca am adauga si operatia DECREMENT la respectivul contor binar (pe ***k*** biti), care este costul mediu pe operatie pentru o secventa de ***n*** operatii (INCREMENT si DECREMENT) succesive?

b) Presupunand ca am adauga si operatia RESET (initializam toti bitii cu 0) la respectivul contor binar (pe ***k*** biti), care este costul mediu pe operatie pentru o secventa de ***n*** operatii (de INCREMENT si RESET) succesive?

Problema 6 - 1p

a) Considerand o structura de date cu operatiile:

- INSERT (S, val)
- DELETE (S)

unde prima operatie insereza intregul *val* in *S*, a doua operatie sterge cele mai mari $|S|/2$ valori din *S*, iar *S* este o lista de intregi ce permite duplicate. Descrieti un mod de implementare a acestei structuri de date astfel incat, orice secventa de n operatii INSERT si DELETE sa fie in timpul O(n).

b) Consideram o structura de date (hashtable) implementata folosind un vector dinamic pentru fiecare bucket. Operatiile definite pentru hashtable sunt INSERT si DELETE. Stergerea presupune cautarea bucket-ului, scoaterea elementului din vectorul corespunzator si shiftarea elementelor urmatoare cu o pozitie la stanga. Inserarea in hashtable presupune gasirea bucket-ului si adaugarea elementului in vectorul corespunzator la sfarsit. Totusi, cand se adauga un element intr-un bucket si vectorul este plin, trebuie sa se aloce un nou vector si sa se copieze elementele din vectorul vechi in cel nou, iar apoi se va folosi vectorul nou pe post de bucket.

ba) Care este complexitatea operatiilor INSERT si DELETE in cazul cel mai defavorabil? Justificati.

bb) Daca dimensiunea vectorului se incrementeaza cu 1 atunci cand este necesar un vector mai mare, care este costul total pentru n operatii si care este costul mediu (amortizat) al unei operatii pe hashtable, folosind metoda agregarii ?

bc) Daca dimensiunea vectorului se dubleaza atunci cand este necesar un vector mai mare, care este costul total pentru n operatii si care este costul mediu (amortizat) al unei operatii pe hashtable, folosind metoda agregarii ?

Problema 7 - 1p

a) Scrieti un algoritm nedeterminist pentru problema: dandu-se un set de ***n*** dreptunghiuri mici de dimensiuni ***(a_i, b_i)***, unde ***$1 \leq i \leq n$*** si un dreptunghi mare de dimensiune ***(A, B)***, poate fi dreptunghiul mare acoperit cu un subset de dreptunghiuri mici, astfel incat sa nu ramana niciun spatiu gol? Oricare doua dreptunghiuri mici din subset sunt distincte si nu trebuie sa se intersecteze atunci cand acopera dreptunghiul mare.

b) Scrieţi un algoritm nedeterminist pentru problema: se da un graf ***G = (V, E)***, astfel incat fiecare nod ***v*** are asociata o pondere intreaga - ***weight(v)***. Se poate gasi un subset de noduri ***$V' \subseteq V$*** astfel incat: intre oricare doua noduri din ***V'*** sa nu exista muchie in ***E***, iar suma ponderilor nodurilor din ***V'*** sa fie cea mai mare posibila ($\sum_{v \in V'} weight(v) = max$)?

Determinati si justificati complexitatea temporala si spatiala pentru cei doi algoritmi.

Problema 8 - 1p

a) Stabiliti o reducere polinomiala intre problema q-sume si problema rucsacului.

b) In ce clasa de dificultate se afla problema rucsacului daca stiti ca problema q-sume este NP-completa?

Problema rucsacului: Se dau n obiecte ce au valoarea v[i] si greutatea w[i] si un rucsac in care se poate transporta o greutate maxima W. Putem transporta cu rucsacul obiecte care sa aiba cel putin valoarea V fara a depasi greutatea W ?
Problema q-sume: Se dau n numere intregi si un intreg q, exista un subset de numere din cele n astfel incat suma acestuia sa fie q?

Observatii:
1) un obiect se ia in rucsac ca intreg, nu fractionar;
2) pentru reducerea polinomiala, puteti introduce si alte constrangeri - de exemplu, limitarea valorii transportate in rucsac

Problema 9 - 1p

a) Aratati ca problema 2-SAT este in P.
Problema 2-SAT: Dandu-se o formula F in forma normala conjunctiva, cu proprietatea ca fiecare disjunctie contine maxim 2 literali, este F satisfabila?

b) Aratati ca problema 3-SAT este NP-completa.
Problema 3-SAT: Dandu-se o formula F in forma normala conjunctiva, cu proprietatea ca fiecare disjunctie contine maxim 3 literali, este F satisfabila?

Problema 10 - 1p

Aratati ca urmatoarea problema este NP-completa: *dandu-se doua grafuri G1 = (V1, E1) si G2 = (V2, E2), sa se gaseasca numarul minim de noduri pe care putem sa le stergem din G1 si din G2 (impreuna cu muchiile incidente in ele) astfel incat subgrafurile rezultate in urma stergerii sa fie identice.*